HAW HAMBURG

BACHELORTHESIS
Asma Abdi

Reference Implementation of secure communication channel for Smart Metering

FACULTY OF ENGINEERING AND COMPUTER SCIENCE
Department of Information and Electrical Engineering

first examiner: Prof. Dr. Kolja Eger
Second examiner: Prof. Dr. Lutz Leutelt

Day of submission: 06. March 2026

HOCHSCHULE FÜR ANGEWANDTE WISSENSCHAFTEN HAMBURG
Hamburg University of Applied Sciences

List of abbreviations

BSI	Bundesamt für Sicherheit in der Informationstechnik
CLS	Controllable Local System
SMGW	Smart Meter Gateway
TLS	Transport Layer Security
HAN	Home Area Network
WAN	Wide Area Network
iMSys	intelligent metering system
LMN	Local Metrological Network
TCP	Transmission Control Protocol
SM-PKI	Smart Metering Public Key Infrastructure

Table Of Figures

Figure 1: General architecture of a proxy server between client and internet. The proxy server mediates communication and prevents direct client access to external resources. [4]..... 7
Figure 2: Use of the TLS proxy function of the SMGW by the CLS communication adapter and CLS component. Arrow outlines indicate physical connections, while green arrows represent a TLS connection. 8
Figure 3: Class diagram – CLI model 21
Figure 4 Sequence Diagram – CLI Proxy Communication..... 25
Figure 5: Class diagram- SRV proxy Model..... 26
Figure 6: Sequence Diagram – SRV proxy communication 28
Figure 7: Class diagram – TLS termination model..... 31
Figure 8: Sequece Diagram – TLS termination 33
Figure 9: backend output 36
Figure 10: SMGW output..... 37
Figure 11: CLS output 37
Figure 12: Wireshark capture of CLI proxy communication. After TLS handshake completion, only TLSv1.3 “Application Data” records are visible. No readable application payload appears in the packet trace, confirming that the SMGW does not access plaintext data. 50

Table of Contents

1. Introduction 5
2. Theory 6
2.1 Smart Metering Infrastructure 6
2.1.1 Proxy 7
2.2 TLS Communication 8
2.3 TLS Proxy Architecture 10
2.4 Certificate Profiles 10
2.5 STRIDE Security Model 11
3. Requirements 13
3.1 Normative Background (Derived BSI) 16
3.2 Functional Requirements of the Reference Implementation 17
3.3 Non-Functional Requirements 19
4. Design 20
4.1 Transparent TLS Proxy – CLI Mode 20
4.2 Transparent TLS Proxy – SRV Mode 26
4.3 TLS Termination Model (Initial Prototype) 30
5. Implementation 35
5.1 Model 1 – CLI 35
5.2 Model 2 – SRV 38
5.3 Model 3 – TLS Termination Prototype 39
6. Test and Evaluation 42
6.1 Functional Verification 42
6.2 STRIDE-Based Security Analysis 43
6.3 Experimental Validation of Confidentiality 49
6.4 Comparative Architectural Evaluation 51
7. Summary 53
8. References 55
Appendix 56

1. Introduction

The increasing integration of decentralized energy systems and microgrids in Germany requires secure and standardized communication within the national smart metering infrastructure. At the core of this infrastructure is the Smart Meter Gateway (SMGW), which acts as a certified communication hub between local energy devices and external market participants.

Control boxes, or Controllable Local Systems (CLS), enable the management of controllable loads such as electric vehicle chargers or heat pumps through the SMGW. Acting as a TLS proxy, the SMGW mediates and authorizes encrypted communication between CLS devices and backend systems while maintaining data integrity and authenticity. A proxy generally functions as an intermediary that forwards data between systems while enforcing security policies [**Error! Reference source not found.**].

The communication architecture and security requirements for this setup are defined by the Federal Office for Information Security (BSI), particularly in Technical Guideline. The TLS proxy channel at the HAN interface enables secure communication between CLS components and backend systems in the WAN while enforcing strict security policies [**Error! Reference source not found.**].

This thesis investigates the TLS communication approach defined by the BSI, focusing on its role in enabling secure, interoperable, and efficient channel establishment. The research includes the design and evaluation of a reference implementation as well as a structured security analysis using the STRIDE methodology. The objective is to assess how a TLS proxy channel can be implemented in compliance with regulatory requirements while ensuring cybersecurity in decentralized energy networks. In this context, the thesis addresses the following guiding question:

“How can a TLS channel setup with the Smart Meter Gateway acting as a proxy be implemented to guarantee end-to-end security and reliable performance between control box and grid operation backend?”

To address this research question, a controlled local test environment is developed and implemented in Python to simulate the communication between CLS, SMGW proxy, and backend components. Furthermore, a structured STRIDE-based threat analysis is

conducted to identify potential security risks. The results form the basis for assessing security, performance, and integration aspects of the proposed solution.

2. Theory

This chapter explains the technical background required to understand the secure communication setup between the Control Box (CLS), the Smart Meter Gateway (SMGW), and the backend system. It provides the foundations for TLS communication, proxy behaviour, smart metering architecture, and the regulatory requirements defined by the BSI [**Error! Reference source not found.**].

2.1 Smart Metering Infrastructure

At the core of the intelligent metering system (iMSys) is the Smart Meter Gateway, which acts as the central security anchor and communication hub of the infrastructure [**Error! Reference source not found.**]. The SMGW is responsible for enforcing authentication, authorization, encryption, and secure routing of communication between connected entities.

According to the BSI architecture, the SMGW physically and logically separates three network domains:

- **Wide Area Network (WAN)** – connects the SMGW to external market participants and backend systems.
- **Local Metrological Network (LMN)** – connects the gateway to metering devices.
- **Home Area Network (HAN)** – connects local energy devices such as CLS components.

The physical and logical separation of these domains ensures that communication between network segments can only occur through the controlled interfaces of the SMGW. This design prevents unauthorized direct access between internal and external systems and establishes the SMGW as the single trusted mediation point.

At the HAN interface, the SMGW provides a TLS proxy function that enables secure communication between CLS components and backend systems located in the WAN [**Error! Reference source not found.**]. This proxy mechanism ensures that communication paths are controlled, authenticated, and encrypted while maintaining strict security boundaries between the network domains.

2.1.1 Proxy

A proxy server is an intermediary system that forwards requests and responses between a client and a target server. It acts as a gateway that can enforce security policies, filter traffic, and control access to network resources. By intercepting communication, a proxy can monitor, restrict, or mediate data exchange between different network domains.

The general principle of a proxy server is illustrated in Figure 1 The proxy mediates communication between the client and the external network and prevents direct access to the target system. [**Error! Reference source not found.**]

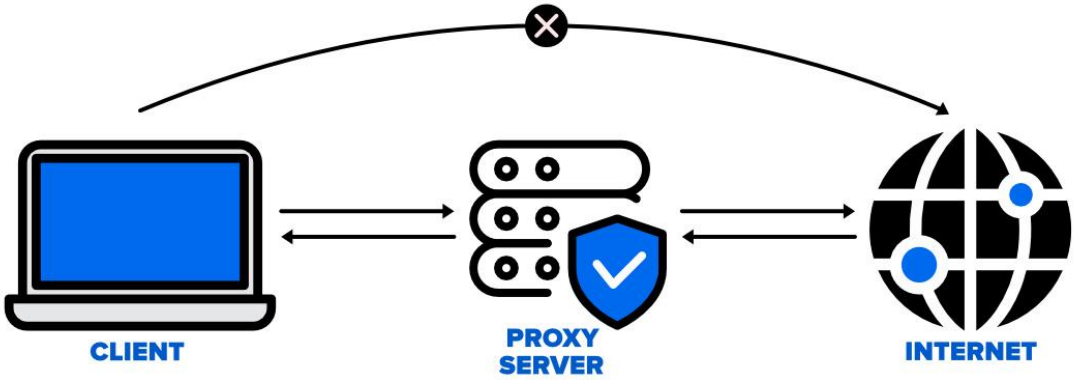


*Figure 1: General architecture of a proxy server between client and internet. The proxy server mediates communication and prevents direct client access to external resources. [**Error! Reference source not found.**]*

While this model describes the generic concept of a proxy, the Smart Meter Gateway implements a specialized TLS proxy function within the smart metering infrastructure. According to the requirements defined, the TLS proxy is provided at the Home Area Network interface of the SMGW and enables secure communication between a CLS and a backend system located in the WAN.

Instead of allowing a direct connection between the CLS and the backend, the SMGW mediates communication across the network boundary. In practice, the realization of the TLS proxy function may follow different architectural approaches. In one approach, the SMGW terminates the TLS session from the CLS and establishes a second independent TLS session toward the backend system. In this case, the SMGW operates as a cryptographic endpoint and processes decrypted application data. Two separate TLS sessions are maintained: one between CLS and SMGW (HAN side) and one between SMGW and backend (WAN side). Alternatively, the SMGW may operate as a transparent mediation component at the transport layer. In this realization, the TLS session is

negotiated end-to-end between CLS and backend, while the SMGW forwards encrypted TLS records without accessing session keys. The gateway enforces authorization and mediation policies but remains outside the cryptographic boundary.

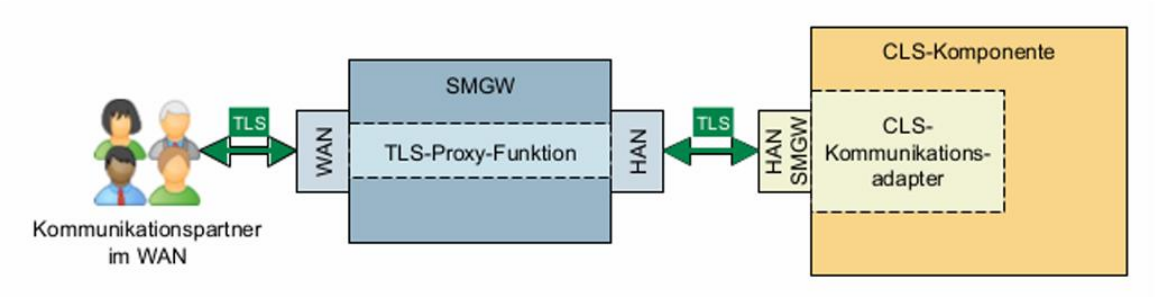


Figure 2: Use of the TLS proxy function of the SMGW by the CLS communication adapter and CLS component. Arrow outlines indicate physical connections, while green arrows represent a TLS connection.

Both approaches satisfy the requirement that communication between HAN and WAN domains must be mediated by the SMGW. However, they differ significantly in terms of cryptographic boundary placement and trusted computing base. These differences are analysed and compared in the later chapters of this thesis.

This architecture prevents unauthorized direct communication across network boundaries and ensures that all data exchange complies with the security policies defined for the intelligent metering system [**Error! Reference source not found.**].

2.2 TLS Communication

Transport Layer Security (TLS) is the primary protocol used to establish secure communication channels over IP-based networks. It operates above the transport layer (TCP) and provides cryptographic protection for application data transmitted between communication partners.

TLS provides three fundamental security properties:

- **Confidentiality** through symmetric encryption
- **Integrity** through message authentication mechanisms
- **Authentication** through certificate-based identity verification

A secure TLS session is established through a handshake procedure. During this process, the communication partners exchange cryptographic parameters, present digital

certificates, validate trust anchors, and derive shared session keys. Only after successful certificate validation and key agreement is encrypted application data transmitted.

Within the smart metering architecture, mutual TLS authentication is required. Both communication partners authenticate each other using certificates. On the HAN side, the CLS component authenticates itself using a CLS_HAN_TLS certificate, while the SMGW presents its corresponding HAN certificate. On the WAN side, certificate-based authentication is also enforced according to the defined certificate profiles.

Depending on the architectural realization of the TLS proxy, TLS sessions may either be terminated at the SMGW or preserved end-to-end between CLS and backend. In a termination model, the SMGW establishes two independent TLS sessions and processes decrypted application data. In a transparent proxy model, the TLS handshake and session key derivation occur directly between CLS and backend, while the SMGW forwards encrypted TLS records without participating in the cryptographic exchange.

The placement of the TLS termination point therefore determines whether the SMGW operates as a cryptographic endpoint or as a transport-level mediation component. This distinction has significant security implications, particularly with regard to confidentiality, integrity, and the size of the trusted computing base [**Error! Reference source not found.**].

Advantages of TLS in Smart Metering Systems

TLS offers several properties that make it particularly suitable for regulated infrastructures such as intelligent metering systems.

First, TLS ensures strong encryption of transmitted data, protecting measurement values and control commands from unauthorized interception. Second, certificate-based mutual authentication enables reliable device identification and prevents unauthorized system integration. Third, TLS supports cryptographic algorithm agility, allowing systems to adapt to updated security recommendations without fundamental architectural changes.

Additionally, TLS operates transparently between the transport and application layers. Once established, the secure channel protects application data without requiring modifications to application logic. Its widespread standardization and mature implementations further contribute to interoperability and long-term maintainability.

In modern networked infrastructures, TLS therefore provides a robust and adaptable foundation for secure communication. Its continuous evolution ensures resilience against emerging cyber threats and supports the long-term security objectives of the smart grid [**Error! Reference source not found.**].

2.3 TLS Proxy Architecture

While Section 2.1.1 introduced the proxy concept at an architectural level, this section describes the TLS proxy mechanism.

The Smart Meter Gateway provides a mediated TLS proxy channel that enables secure communication between a Controllable Local System in the Home Area Network and a backend system in the Wide Area Network. The establishment of this proxy channel can follow different communication modes.

In a client-initiated scenario, the CLS establishes a TLS connection to the SMGW and requests the creation of a proxy channel toward a specified backend system. This is realized through interfaces such as:

- **HKS.TLSPROXY.SOCKSCLI**, which uses a SOCKS-based mechanism for target specification. The detailed SOCKS framing is defined in the specification but is not implemented in this prototype,
- **HKS.TLSPROXY.CLI**, which defines a client-driven proxy channel establishment.

In these modes, the SMGW validates the request, checks authorization policies, and only then establishes a corresponding TLS connection to the backend system. In contrast, the **HKS.TLSPROXY.SRV** mode allows the SMGW to initiate communication toward the CLS.

The normative security and functional requirements associated with these communication modes are already defined. In this thesis, the relevant requirements for each setup are systematically derived and formalized in Chapter 3 (Requirements), where they are translated into concrete implementation specifications.

2.4 Certificate Profiles

Certificate-based authentication is required for establishing TLS connections between the CLS, the Smart Meter Gateway, and backend systems. The specification defines concrete certificate types and validation models that must be used for communication at the HAN and WAN interfaces.

At the HAN interface, the SMGW presents a certificate of type GW_HAN_TLS_CRT. This certificate can appear in two variants:

- CT_Selfsigned: where the certificate is self-signed by the SMGW and may function as a trust anchor in Direct Trust mode.
- CT_SMPKI_Signed: where the certificate is issued by a Sub-CA of the Smart Metering Public Key Infrastructure (SM-PKI). In this case, validation is performed through certificate chain verification against a configured trust anchor [1].

The CLS communication adapter uses a CLS_HAN_TLS_CRT, which is device-specific and used to authenticate the CLS during the TLS handshake at the HAN interface. The validation of this certificate follows the rules defined for the respective trust model.

Backend systems in the WAN also present TLS certificates. Depending on the deployment scenario, these certificates are validated against configured trust anchors and must comply with the structural and usage constraints defined in the specification.

The certificate profiles define mandatory structural properties, including issuer–subject relationships, key usage constraints, and Extended Key Usage (EKU) attributes. Depending on the operational mode, validation may occur either through certificate chain verification (PKI-based trust) or through certificate pinning (Direct Trust).

By defining explicit certificate types and validation mechanisms, the specification ensures that authentication and channel establishment are strictly bound to regulated certificate profiles. Only communication partners presenting certificates that comply with these profiles and validation rules are permitted to establish TLS proxy connections [**Error! Reference source not found.**].

2.5 STRIDE Security Model

The STRIDE model is a structured threat modelling methodology originally introduced by Microsoft to systematically identify security threats in software systems. It classifies threats into six categories based on the type of security violation they represent:

- **Spoofing** – Impersonation of an identity
- **Tampering** – Unauthorized modification of data
- **Repudiation** – Denial of performed actions
- **Information Disclosure** – Unauthorized access to confidential information
- **Denial of Service (DoS)** – Disruption of system availability
- **Elevation of Privilege** – Gaining unauthorized access rights

The STRIDE model is typically applied to system architectures using Data Flow Diagrams (DFDs). Each system component, data store, communication flow, and trust boundary is analysed to determine which STRIDE categories are applicable. This structured mapping ensures that common classes of security weaknesses are systematically considered rather than identified ad hoc.

Each STRIDE category is associated with fundamental security properties:

- Spoofing relates to authentication
- Tampering relates to integrity
- Repudiation relates to accountability
- Information disclosure relates to confidentiality
- Denial of service relates to availability
- Elevation of privilege relates to authorization

By linking threats to core security properties, STRIDE enables a comprehensive and structured evaluation of system security.

In the context of the TLS proxy architecture, STRIDE is used to analyse potential risks in:

- The TLS handshake process
- Certificate validation mechanisms
- CONNECT-based tunnel establishment requests
- Authorization enforcement within the SMGW
- Communication between HAN and WAN domains

This allows the identification of potential weaknesses such as identity spoofing through invalid certificates, manipulation of proxy target requests, unauthorized privilege escalation within communication profiles, or denial-of-service attacks targeting TLS endpoints.

Through systematic threat categorization, STRIDE provides a structured framework for evaluating the security robustness of the implemented communication channel.

Summary of the chapter

The secure communication channel within the smart metering infrastructure is based on several interdependent technical and security mechanisms.

Transport Layer Security provides confidentiality, integrity, and mutual authentication for communication between CLS, SMGW, and backend systems. The Smart Meter Gateway enforces strict separation of network domains and mediates communication through a TLS proxy architecture. Certificate-based authentication ensures that only authorized entities can establish secure channels, while defined certificate profiles regulate structural and usage constraints. The communication behaviour and security mechanisms are governed by the specifications. To evaluate the robustness of this architecture, structured threat modelling using the STRIDE methodology is applied. STRIDE enables systematic identification of potential vulnerabilities across authentication, authorization, integrity, confidentiality, and availability aspects of the system. Together, TLS-based security, proxy mediation, certificate validation, regulatory specifications, and structured threat modelling form the technical and analytical foundation for the design, implementation, and evaluation presented in the subsequent chapters.

3. Requirements

This chapter specifies the requirements for the development of a reference implementation of a secure communication channel for smart metering systems based on the BSI Technical Guideline. The objective is not full certification compliance, but the realization of selected TLS proxy communication scenarios in a controlled test environment. The chapter distinguishes between:

- Normative background requirements derived from BSI
- Concrete functional requirements of the implemented prototypes
- Non-functional requirements and implementation constraints

By defining scope, system functions, constraints, and interfaces, this chapter establishes the foundation for the subsequent design, implementation, and evaluation of the reference system.

- **Scope**

The scope of this implementation comprises the development of three core components that simulate a secure smart metering communication infrastructure:

- A Controllable Local System (CLS) client
- A Smart Meter Gateway (SMGW) TLS proxy
- A grid operation backend server

The reference system implements selected TLS proxy communication scenarios defined in TR-03109-5, specifically:

- HKS.TLSPROXY.CLI
- HKS.TLSPROXY.SRV

The standardized HKS.TLSPROXY.SOCKSCLI scenario and its associated SOCKS-TLS framing defined in the detail specification are explicitly outside the scope of this work.

In the implemented transparent proxy models, the SMGW mediates TCP connections and enforces authorization policies while preserving end-to-end TLS between CLS and backend. In addition, an experimental TLS termination model is implemented for comparative architectural and security evaluation.

The reference implementation serves as:

- A proof-of-concept for selected TLS proxy architectures
- A controlled environment for structured security analysis using the STRIDE methodology
- An analytical comparison between transparent TLS proxying and TLS termination approaches

The implementation is not intended to represent a certification-conform communication adapter and does not claim full compliance with TR-03109-5. Instead, it derives its architectural principles from the guideline and evaluates their security implications in a controlled experimental setting.

- **Product Perspective**

The implementation represents a reference system for selected TLS proxy communication scenarios derived from the architectural principles. It models the interaction between a Controllable Local System (CLS), a Smart Meter Gateway (SMGW), and a backend system within a controlled virtual test environment.

The focus of the implementation lies on correct realization of TLS-based secure communication, architectural comparison between transparent proxying and TLS termination, and systematic security evaluation using the STRIDE methodology. The system is designed as an analytical prototype and does not aim at production deployment or certification conformity with TR-03109-5.

- **Product Functions**

The system provides the following core functionality:

1. Establish TCP-based communication between CLS, SMGW, and backend.
2. Implement transparent TLS proxy communication (CLI mode).
3. Implement transparent TLS proxy communication (SRV mode).
4. Implement an experimental TLS termination architecture.
5. Perform mutual TLS authentication between communication partners.
6. Validate certificates against configured trust anchors.
7. Forward encrypted TLS records transparently in proxy mode.
8. Decrypt and re-encrypt application data in termination mode.
9. Enforce whitelist-based backend authorization.
10. Provide logging for functional verification and security analysis.

- **Constraints**

The implementation is subject to the following constraints:

1. Operation within a local virtual test environment.
2. Use of Python 3.x for all components.
3. Command-line execution of system components.
4. Use of self-signed certificates for testing.
5. No implementation of SOCKS-TLS framing.

- **Assumptions and Dependencies**

The following assumptions apply:

1. Test certificates are pre-generated and securely distributed between components.

2. Local network connectivity between components is available.
3. System clocks are sufficiently synchronized to allow certificate validity checks.
4. The simulated SMGW behaves according to the normative requirements of BSI

- **External Interface Requirements**

The system operates within a local TCP-based environment. Communication between components is realized through defined TCP ports. In transparent proxy mode, the SMGW mediates TCP connections between CLS and backend without terminating TLS. In termination mode, the SMGW acts as a TLS endpoint on both communication sides. Certificates are stored locally in PEM format and loaded at runtime. No external certificate authority services are integrated.

3.1 Normative Background (Derived BSI)

The BSI defines requirements for CLS communication adapters connected to the Smart Meter Gateway. The following architectural principles form the normative foundation of this work:

ID	Priority	Requirement
REQ-NORM-1	MUST	The implementation SHALL use the TLS proxy function provided at the logical interface IF_GW_CLS.
REQ-NORM-2	MUST	The CLS communication adapter SHALL terminate or participate in the TLS proxy channel at the HAN side.
REQ-NORM-3	MUST	The SMGW SHALL mediate a TLS-secured proxy channel between CLS (HAN) and a communication partner in the WAN.

Selected Communication Scenarios

The following HKS scenarios are implemented:

ID	Priority	Requirement
REQ-HKS-1	MUST	The reference implementation SHALL realize HKS.TLSPROXY.CLI.
REQ-HKS-2	MUST	The reference implementation SHALL realize HKS.TLSPROXY.SRV.

REQ-HKS-3	WON'T	HKS.TLSPROXY.SOCKSCLI WILL NOT be implemented in this reference system.
REQ-HKS-4	MUST	Each HKS SHALL be identifiable via TCP port number and TLS parameters.

The focus of this thesis lies on analysing architectural and security properties of transparent TLS proxy communication. The SOCKS-TLS framing defined in the detail specification is outside the scope of this prototype.

3.2 Functional Requirements of the Reference Implementation

The following requirements define the behaviour of the implemented prototype.

CLS Component

ID	Priority	Requirement
REQ-CLS-1	MUST	The CLS SHALL initiate a TCP connection to the SMGW (CLI mode).
REQ-CLS-2	MUST	The CLS SHALL support accepting TCP connections from the SMGW (SRV mode).
REQ-CLS-3	MUST	The CLS SHALL transmit a TCP-based CONNECT control message specifying the backend target.
REQ-CLS-4	MUST	The CLS SHALL initiate a TLS handshake toward the backend after tunnel establishment (transparent proxy mode).
REQ-CLS-5	MUST	The CLS SHALL enforce mutual TLS authentication.
REQ-CLS-6	SHOULD	The CLS SHOULD log TLS handshake and tunnel negotiation events.

SMGW Proxy Component – Transparent Proxy Mode

ID	Priority	Requirement
REQ-SMGW-1	MUST	The SMGW SHALL accept incoming TCP connections at the HAN interface.
REQ-SMGW-2	MUST	The SMGW SHALL validate requested backend targets against a whitelist.

REQ-SMGW-3	MUST	The SMGW SHALL establish a TCP connection to the backend.
REQ-SMGW-4	MUST	The SMGW SHALL forward raw TCP payload bidirectionally without TLS termination (CLI and SRV mode).
REQ-SMGW-5	SHOULD	The SMGW SHOULD log connection establishment and validation events.

In transparent proxy mode, the SMGW does not terminate TLS. The TLS session is negotiated end-to-end between CLS and backend, and the SMGW does not access cryptographic key material.

SMGW – TLS Termination Prototype

A third model is implemented for comparative security analysis (first prototype). This model is explicitly marked as experimental and is not aligned with transparent TLS proxy behaviour.

ID	Priority	Requirement
REQ-TERM-1	MUST	The SMGW SHALL terminate the TLS session from CLS (HAN side).
REQ-TERM-2	MUST	The SMGW SHALL establish a second independent TLS session toward the backend.
REQ-TERM-3	MUST	The SMGW SHALL decrypt and re-encrypt application data between both TLS sessions.

Certificate and Authentication Requirements

ID	Priority	Requirement
REQ-CERT-1	MUST	Mutual TLS authentication SHALL be implemented.
REQ-CERT-2	MUST	Certificates SHALL be validated against configured trust anchors.
REQ-CERT-3	MUST	TLS handshake SHALL be aborted upon validation failure.
REQ-CERT-4	SHOULD	CT_Selfsigned (Direct Trust) SHALL be supported in the local test environment.

3.3 Non-Functional Requirements

Security

ID	Priority	Requirement
REQ-SEC-1	MUST	TLS encryption SHALL protect communication channels.
REQ-SEC-2	MUST	Private keys SHALL not be exposed via interfaces.

Performance

ID	Priority	Requirement
REQ-PERF-1	SHOULD	TLS handshake latency SHALL be measurable.
REQ-PERF-2	SHOULD	Round-trip time SHALL be measurable.

Implementation Constraints

ID	Priority	Constraint
REQ-CON-1	MUST	Implementation SHALL operate in a local virtual environment.
REQ-CON-2	MUST	Python 3.x SHALL be used.
REQ-CON-3	SHOULD	Predefined local TCP ports SHOULD be used.
REQ-CON-4	WON'T	Integration with official SM-PKI WILL NOT be implemented.
REQ-CON-5	WON'T	Certificate revocation checking WILL NOT be implemented.

4. Design

This section describes the architectural design and modelling of three communication approaches implemented in this thesis:

1. TLS Proxy – CLI Mode
2. TLS Proxy – SRV Mode
3. TLS Termination (Initial Prototype made)

The models differ in connection initiation and placement of the cryptographic boundary. The design follows the Smart Meter Gateway communication architecture. Each design has its own sequence diagram and UML. Since TLS Proxy – SOCKSCLI mode was not implemented, the design discussion focuses on the three realized architectures.

4.1 Transparent TLS Proxy – CLI Mode

Architectural Overview

The CLI scenario models a client-initiated TLS proxy architecture in which the Controllable Local System establishes a communication channel through the Smart Meter Gateway toward a backend system located in the Wide Area Network.

In this design, the CLS actively initiates the communication process. The SMGW acts as a mediation component that enforces authorization policies and establishes a forwarding tunnel. However, the SMGW does not terminate the TLS session and does not possess TLS session keys. Instead, it forwards encrypted TLS records transparently between CLS and backend.

The cryptographic boundary is therefore located strictly between CLS and backend. The SMGW operates as a transport-level proxy and mediation point, but not as a TLS endpoint.

This model reflects the proxy principle defined in the smart metering architecture, where communication between network domains must pass through the SMGW while maintaining secure channel properties.

Structural Design (Class Diagram)

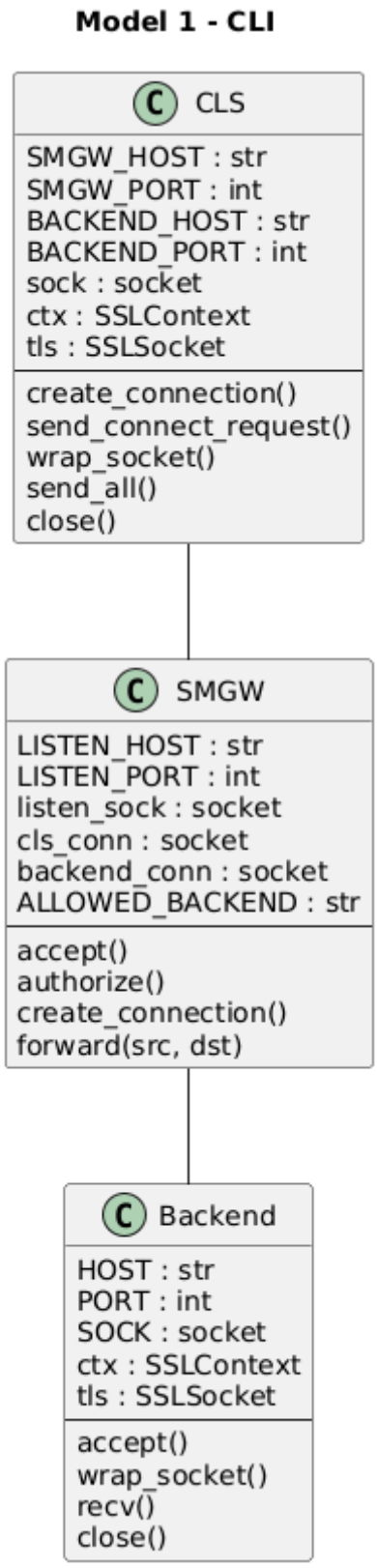


Figure 3: Class diagram – CLI model

The system consists of three logical components (see figure 3):

- CLS
- SMGW
- Backend

The CLS class contains attributes for the SMGW address, backend target address, TCP socket, SSL context, and TLS socket. Its responsibilities include establishing the TCP connection to the SMGW, initiating tunnel establishment, performing the TLS handshake, and transmitting encrypted application data.

The SMGW class models the mediation function. It maintains listening socket attributes, connection sockets for both CLS and backend, and a whitelist configuration for allowed backend targets. Its responsibilities include accepting incoming CLS connections, validating the requested backend target, establishing a TCP connection to the backend, and forwarding encrypted data in both directions.

The Backend class models the WAN communication partner. It maintains its listening configuration and TLS context and is responsible for terminating the TLS session and processing decrypted application data.

Associations between these classes are modelled as undirected to reflect communication relationships rather than structural ownership or navigability.

Behavioural Design (Sequence Diagram)

Figure 4 presents the sequence diagram of the CLI proxy communication. The interaction can be divided into four logically phases, which helps with explaining the concept.

- **Phase 1 – TCP Connection Establishment**

The communication begins with the CLS establishing a TCP connection to the SMGW. This step creates a transport-layer channel but does not yet activate TLS. The SMGW accepts the incoming connection and activates its corresponding lifeline. At this stage, communication is purely TCP-based and unencrypted.

- **Phase 2 – Control Phase**

After TCP establishment, the CLS sends a custom control message of the form:

```
CONNECT <backend_host> <backend_port>
```

In the simulation environment, this corresponds to:

```
CONNECT 127.0.0.1 9443
```

This message is transmitted over the established TCP channel and represents a simplified control instruction for tunnel establishment within the prototype. It does not implement the standardized SOCKS-TLS framing defined in the TR-03109-5 details specification, which is outside the scope of this work.

Upon receiving this request, the SMGW performs a validation step. This step corresponds to an access control check in which the requested backend host is compared against a predefined whitelist. If the target is not authorized, the request is rejected and the connection is terminated.

If validation succeeds, the SMGW establishes a new TCP connection toward the backend endpoint in the WAN domain. After successful connection, the SMGW confirms tunnel establishment to the CLS.

At the end of this phase, a bidirectional TCP tunnel exists between CLS and backend, mediated by the SMGW.

- **Phase 3 – End-to-End TLS Handshake**

After successful tunnel establishment, the CLS initiates a TLS handshake. The TLS handshake messages are transmitted through the established TCP tunnel.

The SMGW does not interpret or modify these messages. Instead, it forwards raw TCP payloads between the CLS and backend sockets.

The backend receives the TLS ClientHello and responds with the appropriate ServerHello, certificate, and handshake parameters. Mutual authentication is enforced through certificate validation during the TLS handshake.

The TLS session is therefore negotiated directly between CLS and backend. Session keys are derived exclusively at these endpoints. The SMGW has no access to the cryptographic material and cannot decrypt the transmitted data.

This phase establishes confidentiality, integrity, and endpoint authentication.

- **Phase 4 – Encrypted Application Data Exchange**

After successful handshake completion, application data is transmitted from CLS to backend through the TLS session.

Although application payload appears as plaintext at the application layer (e.g., measurement values), it is encrypted into TLS application records before transmission over TCP.

The SMGW forwards encrypted TLS records without inspection. Decryption occurs only at the backend TLS endpoint.

The backend processes the decrypted application data and returns a response. This response is encrypted at the backend, forwarded by the SMGW, and decrypted at the CLS. Throughout this phase, the SMGW remains outside the cryptographic boundary.

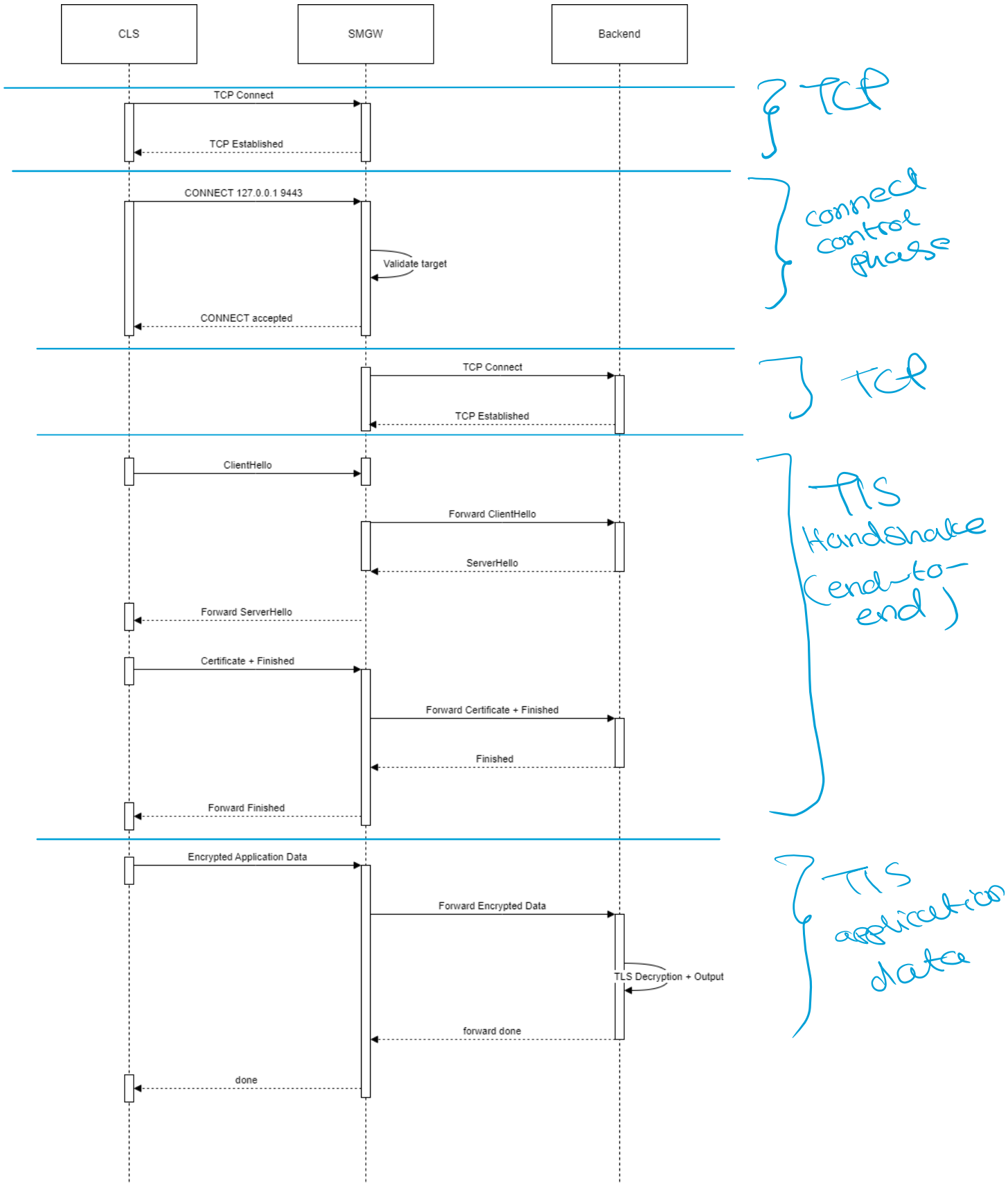


Figure 4: Sequence Diagram – CLI Proxy Communication

4.2 Transparent TLS Proxy – SRV Mode

Architectural Overview

The SRV scenario represents the server-initiated variant of the TLS proxy architecture. In contrast to the CLI model, the SMGW initiates the TCP connection toward the CLS. Despite this inversion of connection initiation, the cryptographic properties of the system remain unchanged. The SMGW establishes a TCP connection to a service port exposed by the CLS within the Home Area Network (HAN). After the tunnel establishment phase, the TLS handshake is performed end-to-end between CLS and backend. The SMGW continues to act as a transparent forwarding entity and does not terminate the TLS session. Thus, as in the CLI model, the cryptographic boundary remains strictly between CLS and backend. The SMGW performs mediation and forwarding but does not participate in cryptographic key derivation or payload decryption. The primary architectural difference between CLI and SRV mode lies in the direction of connection initiation.

Structural Design (Class Diagram)

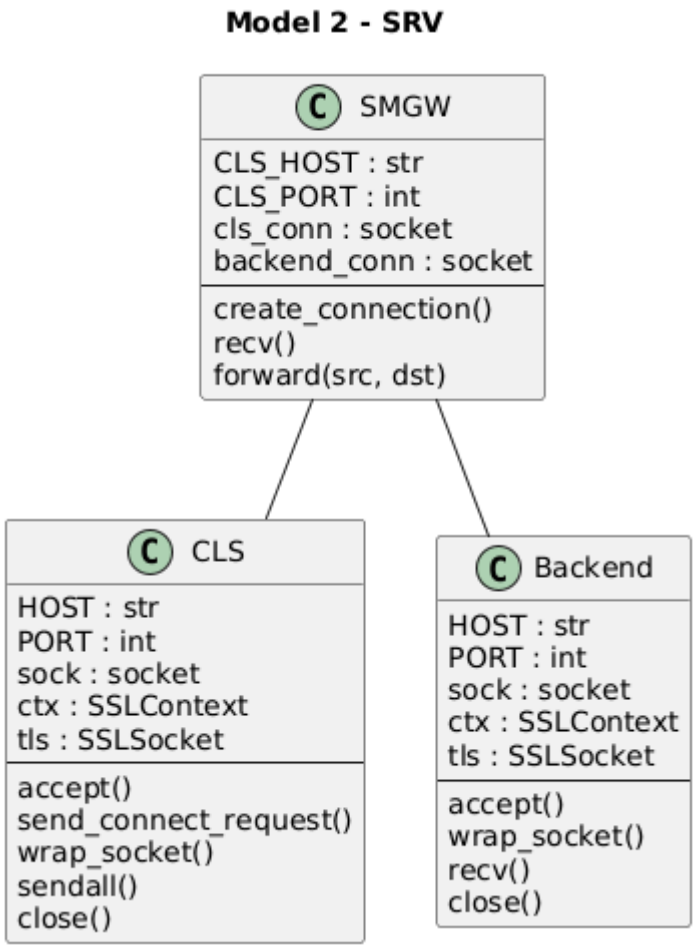


Figure 5: Class diagram- SRV proxy Model

The structural composition of the SRV model closely resembles the CLI architecture. The same three logical components exist:

- CLS
- SMGW
- Backend

However, the roles during connection establishment differ. The CLS class exposes a listening service port and accepts incoming connections from the SMGW. It maintains socket and TLS context attributes similar to the CLI model but includes server-side connection handling logic. The SMGW class initiates the TCP connection toward the CLS and receives the CONNECT control message. After validating the requested backend target, it establishes a TCP connection toward the backend and switches to forwarding mode. The Backend class remains identical to the CLI scenario. It acts as the TLS endpoint and performs decryption of application data. As in the CLI model, associations between components are modelled as undirected.

Behavioural Design (Sequence Diagram)

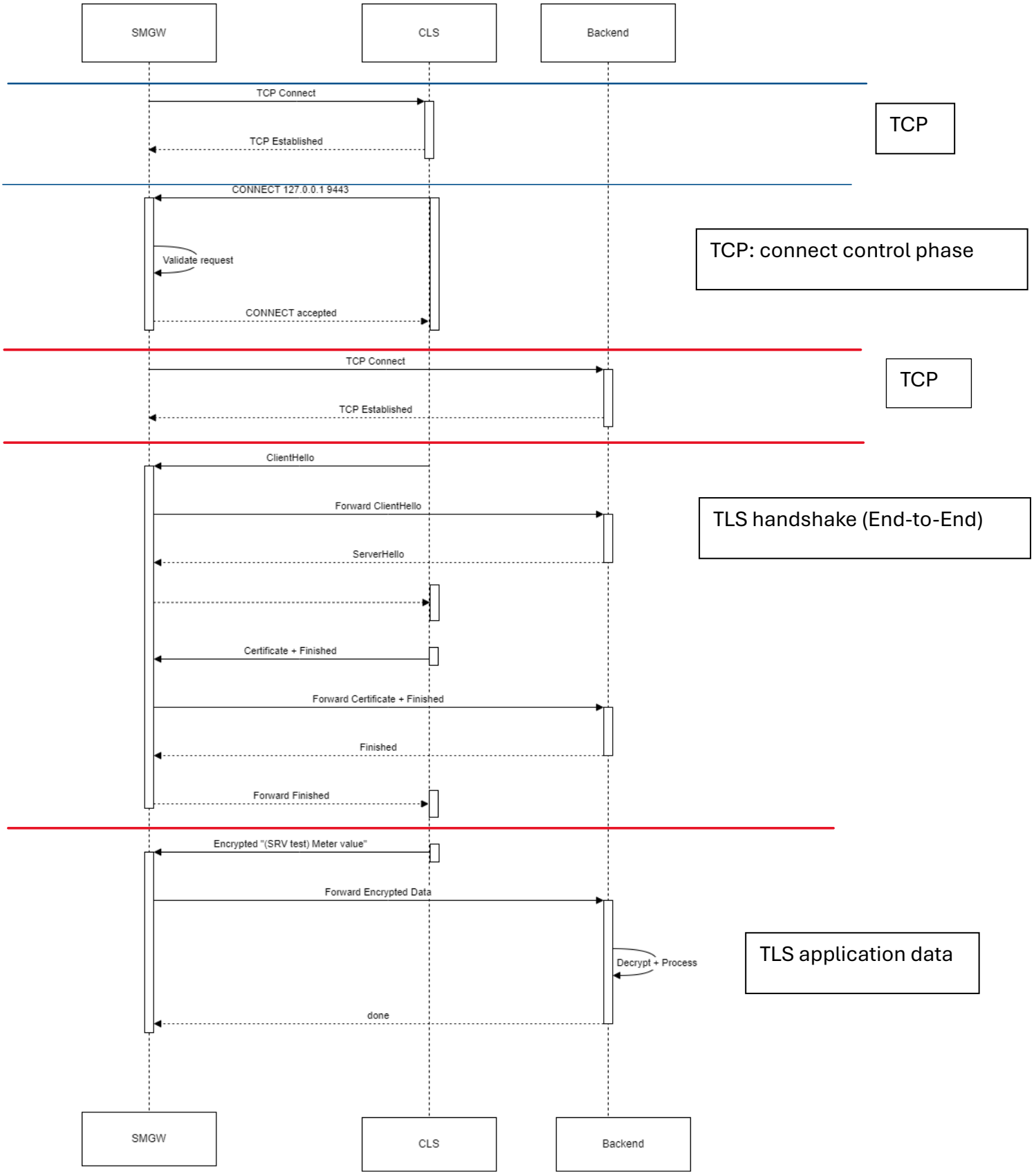


Figure 6: Sequence Diagram – SRV proxy communication

The SRV sequence diagram consists of four major phases, similar to the CLI model, with reversed initiation logic.

- **Phase 1 – TCP Connection Establishment (SMGW → CLS)**

The sequence begins with the SMGW actively establishing a TCP connection to the CLS service port. This reverses the initiation direction compared to CLI mode. At this stage, communication remains unencrypted and operates solely at the transport layer (TCP). The purpose is to create the channel required for subsequent control negotiation.

- **Phase 2 – Control Phase (Tunnel Establishment)**

After accepting the TCP connection, the CLS transmits a CONNECT control message specifying the backend target:

```
CONNECT 127.0.0.1 9443
```

This message is identical in structure to the CLI model and represents a simplified application-layer tunnel establishment command. The SMGW receives the request and performs a validation step. As in CLI mode, this corresponds to a whitelist-based access control check that ensures the requested backend is authorized. If validation succeeds, the SMGW establishes a TCP connection toward the backend endpoint in the WAN domain. Upon successful connection, the SMGW transitions into transparent forwarding mode. At the end of this phase, a bidirectional TCP tunnel exists between CLS and backend.

- **Phase 3 – End-to-End TLS Handshake**

Once the TCP tunnel is established, the CLS initiates the TLS handshake. The TLS negotiation occurs directly between CLS and backend. The SMGW forwards TLS handshake messages without interpretation. It does not terminate TLS and does not access session keys.

The handshake establishes:

- Mutual authentication via certificates
- Session key derivation
- Confidential communication channel

As in the CLI model, the TLS session is negotiated exclusively between CLS and backend.

- **Phase 4 – Encrypted Application Data Exchange**

After handshake completion, encrypted TLS application records are exchanged between CLS and backend. The SMGW forwards encrypted data transparently. Decryption occurs only at the backend. Responses are encrypted at the backend and forwarded unchanged to the CLS.

4.3 TLS Termination Model (Initial Prototype)

Architectural Overview

The TLS termination model represents the initial prototype developed during early experimentation. In contrast to the transparent proxy models, the SMGW acts as a TLS endpoint on both sides of the communication.

In this design:

1. A TLS session is established between CLS and SMGW (HAN side).
2. A second TLS session is established between SMGW and backend (WAN side).

The SMGW decrypts application data received from the CLS and re-encrypts it before forwarding it to the backend.

Structural Design (Class Diagram)

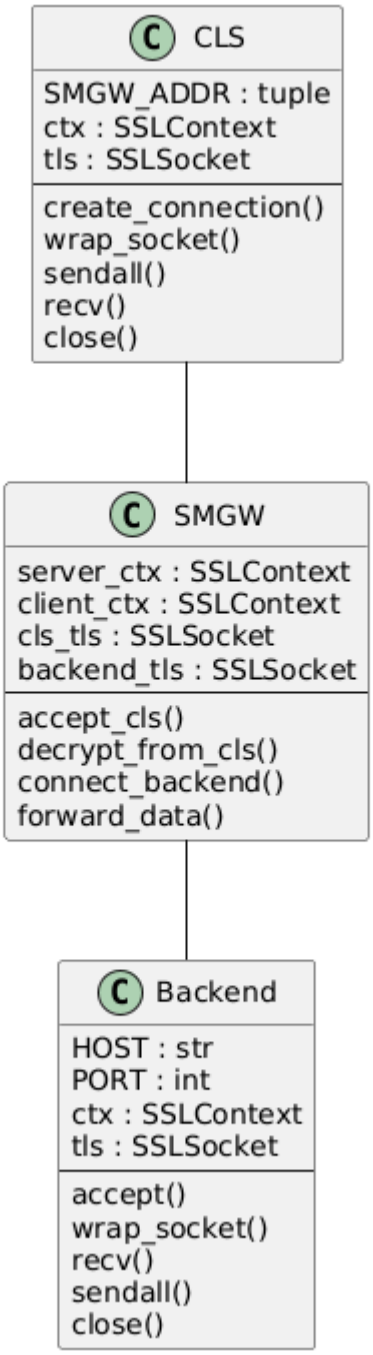


Figure 7: Class diagram – TLS termination model

The structural difference of this model lies primarily within the SMGW component. The SMGW maintains two separate SSL contexts:

- A server-side context for TLS termination from CLS
- A client-side context for TLS establishment toward backend

It also maintains two independent TLS sockets:

- cls_tls
- backend_tls

Unlike the proxy models, the SMGW processes plaintext data. The CLS and Backend components remain TLS endpoints but no longer share a single end-to-end TLS session. Associations remain undirected, representing communication relationships.

Behavioural Design (Sequence Diagram)

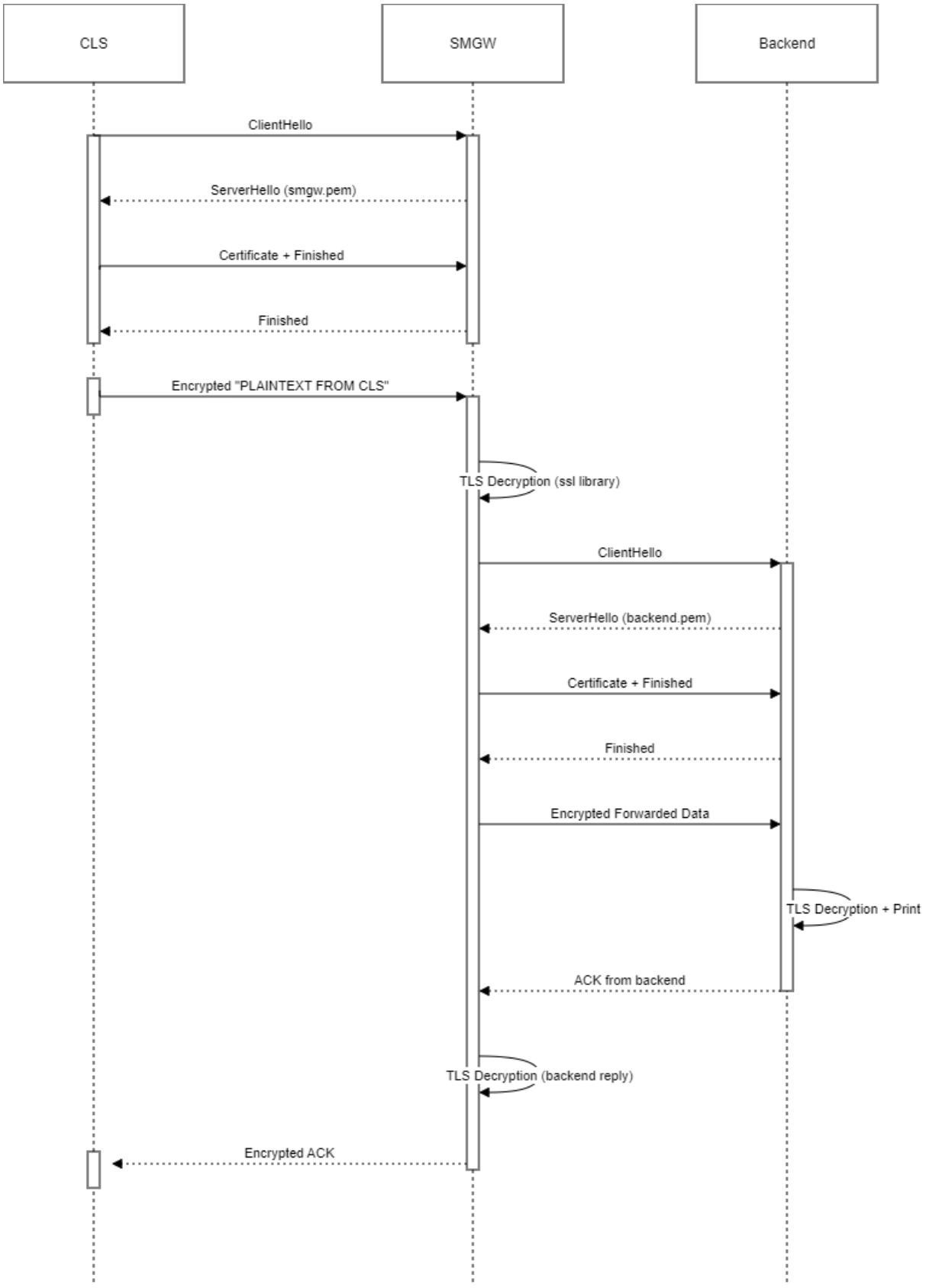


Figure 8: Sequence Diagram – TLS termination

The TLS termination sequence diagram shows two distinct handshake processes.

- **Phase 1 – TLS Handshake (CLS ↔ SMGW)**

The CLS establishes a TCP connection and immediately initiates a TLS handshake toward the SMGW. The SMGW acts as TLS server and terminates the handshake.

At this point, session keys are derived between CLS and SMGW and the SMGW can decrypt application data.

- **Phase 2 – Application Data Decryption**

After handshake completion, the CLS sends encrypted TLS application data. The SMGW decrypts this data locally, resulting in plaintext exposure within the SMGW process. This represents a significant architectural difference compared to the proxy models.

- **Phase 3 – TLS Handshake (SMGW ↔ Backend)**

The SMGW then establishes a second TCP connection to the backend and initiates a new TLS handshake. This handshake is independent of the first one. Separate session keys are derived.

- **Phase 4 – Re-Encryption and Forwarding**

The SMGW re-encrypts the previously decrypted plaintext using the second TLS session and forwards it to the backend. The backend decrypts the data and processes it normally.

Responses follow the reverse path:

- Backend encrypts
- SMGW decrypts
- SMGW re-encrypts
- CLS decrypts

Two cryptographic contexts are therefore active simultaneously.

5. Implementation

This chapter describes the technical realization of the architectural models presented in Chapter 4. While Chapter 4 focuses on conceptual design and protocol behaviour, this chapter explains how the models were implemented in Python within a controlled test environment.

The reference system consists of three independently executed components: CLS, SMGW proxy, and Backend server. All components communicate via TCP sockets and use the Python SSL library to establish TLS-secured channels.

- **Development Environment**

The reference implementation was developed using Python 3.x (later versions can be used) and executed within a controlled local test environment. Communication between components is realized via TCP sockets using the Python socket module, while TLS functionality is provided by the Python ssl library.

All components communicate over localhost (127.0.0.1). Self-signed certificates are used to simulate certificate-based authentication. Integration with the Smart Metering Public Key Infrastructure (SM-PKI) and certificate revocation checking are not implemented, as defined in the constraints of Chapter 3.

The SMGW proxy listens TCP port, while the backend server listens on a defined TLS port. All certificates are stored in PEM format and loaded at runtime into SSL contexts. All components are executed via command-line interfaces.

5.1 Model 1 – CLI

The CLI implementation realizes the architecture described in Section 4.1.

CLS implementation

The CLS component establishes a TCP connection toward the SMGW and sends a custom CONNECT request to initiate tunnel establishment. After successful negotiation, the existing TCP socket is wrapped with an SSL context using:

cls.py snippet code
ctx.wrap_socket(sock, server_hostname="backend")

This initiates the end-to-end TLS handshake toward the backend. From this point onward, all transmitted data is encapsulated in TLS records. The CLS loads its certificate and private key for mutual authentication and validates the backend certificate against a configured trust anchor.

SMGW Implementation

The SMGW proxy accepts incoming TCP connections at the HAN interface. After receiving the CONNECT request, it performs a whitelist-based validation of the requested backend target. If validation succeeds, the SMGW establishes a new TCP connection toward the backend.

The forwarding mechanism is implemented using two parallel threads:

- One thread forwards data from CLS to backend.
- The other thread forwards data from backend to CLS.

The forwarding function relays raw TCP buffers without inspection. The SMGW does not create or manage any TLS context in CLI mode and does not access cryptographic material.

Backend Implementation

The backend listens for TCP connections from the SMGW and wraps accepted sockets using:

Backend.py snippet code
ctx.wrap_socket(conn, server_side=True)

This terminates the TLS handshake initiated by the CLS. The backend decrypts incoming TLS application records and processes the resulting plaintext. Mutual TLS authentication is enforced by requiring client certificates and validating them against the configured trust anchor.

Backend output

```
PS C:\smart-meter-thesis\CLI> py .\backend_server.py
Backend: waiting for CLS via SMGW...
Backend: End-to-end TLS handshake successful
Backend received: (test) Meter value = 123.45 kWh
PS C:\smart-meter-thesis\CLI>
```

Figure 9: backend output

SMGW output

```
PS C:\smart-meter-thesis\CLI> py .\smgw_proxy.py
SMGW: Listening for CLI proxy requests...
SMGW: CLS connected from ('127.0.0.1', 59703)
SMGW: Authorizing tunnel to 127.0.0.1:9443
SMGW: Tunnel established, switching to forwarding mode
PS C:\smart-meter-thesis\CLI>
```

Figure 10: SMGW output

CLS output

```
PS C:\smart-meter-thesis>  cd .\CLI\
PS C:\smart-meter-thesis\CLI> py .\cls_server.py
CLS: Proxy CONNECT request sent
CLS: End-to-end TLS handshake successful
PS C:\smart-meter-thesis\CLI>
```

Figure 11: CLS output

Figures 9–11 show the console outputs of the backend, SMGW, and CLS components. The console output demonstrates the successful establishment of the end-to-end TLS connection between the CLS and the backend system through the SMGW proxy. The CLS initiates the TLS handshake through the established TCP tunnel, while the SMGW forwards encrypted TLS records without participating in the cryptographic exchange.

The SMGW output shows that the CONNECT request from the CLS is accepted and a TCP tunnel is established toward the backend. Since the SMGW operates purely as a forwarding proxy, it does not terminate TLS and therefore does not display decrypted application data. The backend output confirms that the TLS handshake was completed successfully and that the encrypted message transmitted by the CLS was correctly received and decrypted.

The components must be executed in the following order:

1. backend.py
2. smgw.py
3. cls.py

This confirms that the CLI implementation correctly realizes the transparent TLS proxy architecture described in Section 4.1.

5.2 Model 2 – SRV

The SRV implementation follows the design described in Section 4.2. The primary difference from CLI mode lies in connection initiation. The SMGW initiates the TCP connection toward the CLS service port.

CLS Component

In SRV mode, the CLS listens on a defined service port and accepts connections from the SMGW. After accepting the TCP connection, the CLS sends the CONNECT request and initiates the TLS handshake toward the backend. The TLS wrapping procedure and certificate handling remain identical to CLI mode.

SMGW Component (SRV Mode)

The SMGW actively connects to the CLS and waits for the CONNECT request. After parsing and validating the requested backend target, it establishes the TCP connection to the backend. The same forwarding logic as in CLI mode is reused. TLS remains end-to-end between CLS and backend. No cryptographic termination occurs at the SMGW.

Output

The console output confirms the successful establishment of the communication channel between CLS, SMGW, and backend in SRV mode. The backend output shows that the server accepts the TCP connection forwarded by the SMGW and successfully completes the TLS handshake. After the handshake, the backend receives the transmitted test message containing the simulated meter value. The CLS output indicates that the CLS waits for an incoming connection from the SMGW. Once the SMGW connects, the TLS handshake toward the backend is successfully completed. The SMGW output shows that the gateway first establishes a TCP connection to the CLS service port and then connects to the backend system. After both connections are available, the SMGW switches to forwarding mode and relays encrypted TLS traffic between the two endpoints. The observed behaviour is therefore similar to Model 1. Although the direction of the initial TCP connection differs, the TLS session remains end-to-end between CLS and backend, and the SMGW continues to operate purely as a forwarding proxy. The components must be executed in the following order:

1. backend.py
2. cls.py
3. smgw.py

Screenshots of the console outputs are provided in the Appendix.

5.3 Model 3 – TLS Termination Prototype

The TLS termination prototype implements the architectural model described in Section 4.3. Unlike the transparent proxy models, the SMGW acts as an active TLS endpoint on both communication sides. Two independent TLS sessions are created: one between CLS and SMGW and another between SMGW and the backend. The SMGW decrypts and re-encrypts application data, effectively acting as a TLS bridge. The following subsections describe the technical realization in detail.

CLS Implementation

In the termination prototype, the CLS establishes a TCP connection directly to the SMGW and immediately wraps the socket with a TLS context:

CLS.py snippet code
sock = socket.create_connection(SMGW_ADDR) tls = ctx.wrap_socket(sock, server_hostname="smgw")

The TLS handshake is therefore performed between CLS and SMGW.

The CLS loads its client certificate and private key, validates the SMGW certificate against the configured trust anchor, completes mutual TLS authentication and sends encrypted application data over the established TLS channel. From the perspective of the CLS, the SMGW behaves as the final TLS endpoint.

SMGW Implementation

The SMGW component implements two separate SSL contexts which are a server-side context for accepting TLS connections from the CLS and a client-side context for establishing TLS connections to the backend.

Step 1 – TLS Termination

The SMGW listens on a defined HAN port and wraps the accepted socket:

Smgw.py snippet code
cls_tls = server_ctx.wrap_socket(cls_conn, server_side=True)

At this point, the SMGW terminates the TLS handshake, session keys are derived between CLS and SMGW and encrypted data from CLS can be decrypted locally. When the CLS sends application data, the SMGW receives plaintext after decryption:

Smgw.py snippet code
data = cls_tls.recv(4096) print("SMGW DECRYPTED from CLS:", data.decode())

The SMGW now has access to the application payload in unencrypted form.

Step 2 – Establishment of Second TLS Session

After receiving plaintext data, the SMGW establishes a new TCP connection toward the backend:

Smgw.py snippet code
backend_sock = socket.create_connection(BACKEND_ADDR) backend_tls = client_ctx.wrap_socket(backend_sock, server_hostname="backend")

This initiates a second, independent TLS handshake between SMGW and backend. The backend authenticates the SMGW using mutual TLS. Session keys are derived independently from the first TLS session.

Step 3 – Re-Encryption and Forwarding

The SMGW forwards the previously decrypted plaintext to the backend:

Smgw.py snippet code
backend_tls.sendall(data)

The backend decrypts this data using its TLS context and processes it. Responses follow the reverse path:

- Backend encrypts response.
- SMGW decrypts response.
- SMGW re-encrypts response for CLS.
- CLS decrypts response.

The SMGW therefore performs cryptographic operations on both sides.

Output

The console output demonstrates that two independent TLS sessions are established in the termination prototype. First, the TLS handshake is performed between the CLS and the SMGW. After this handshake is completed, the SMGW decrypts the received TLS records and obtains the application data in plaintext.

The SMGW then establishes a second TLS connection toward the backend system and performs another TLS handshake. The previously decrypted plaintext message is re-encrypted and forwarded to the backend through this second TLS session. Unlike Model 1 and Model 2, the SMGW is able to display the decrypted application message in its console output. This confirms that the SMGW terminates the TLS session and processes application data internally. Packet-level analysis further confirms that two separate TLS channels exist: one between CLS and SMGW and one between SMGW and backend. As a result, end-to-end encryption between CLS and backend is no longer preserved.

The components must be executed in the following order:

1. backend.py
2. smgw.py
3. cls.py

Screenshots of the console outputs for this model are included in the Appendix.

6. Test and Evaluation

This chapter evaluates the implemented reference system with respect to the functional and security requirements defined in Chapter 3. The evaluation consists of:

1. Functional verification of communication behaviour
2. Structured threat modelling using STRIDE
3. Experimental validation of confidentiality properties
4. Comparative architectural security assessment

All tests were performed in a controlled local test environment using the Python-based implementation described in Chapter 5.

6.1 Functional Verification

Test Case 1 – Successful Proxy Communication

To verify correct functional behaviour, the backend component was started first, followed by the SMGW proxy and finally the CLS component. This startup order reflects the intended operational sequence in which the communication endpoint and mediation component are available before the CLS initiates communication.

During execution, the CONNECT negotiation phase was successfully completed in both transparent proxy models. The SMGW validated the requested backend target and established the corresponding TCP tunnel. Subsequently, the TLS handshake was completed without errors, and encrypted application data was transmitted through the proxy channel.

In all three implemented architectures — Transparent TLS Proxy (CLI), Transparent TLS Proxy (SRV), and TLS Termination Prototype — application messages were successfully delivered to the backend system and responses were returned to the CLS.

This confirms correct realization of:

- Tunnel establishment logic
- TLS handshake negotiation
- Mutual authentication
- Encrypted application data exchange

The test demonstrates that all implemented models establish operational TLS-secured communication channels as specified in Chapter 3.

Test Case 2 – Certificate Validation Failure

To verify proper enforcement of authentication requirements, a negative test case was conducted. The trusted certificate at one communication endpoint was replaced with an invalid certificate that did not match the configured trust anchor.

Upon execution, the TLS handshake failed during certificate validation. The connection was immediately terminated, and no application data was transmitted.

This behaviour was consistently observed across all three architectural models.

The result confirms correct enforcement of:

- Mutual TLS authentication
- Trust anchor validation
- Immediate handshake termination upon validation failure

Therefore, the implementation satisfies the normative authentication requirements derived in Chapter 3 and demonstrates proper rejection of unauthorized communication partners.

6.2 STRIDE-Based Security Analysis

The security evaluation was performed using the STRIDE methodology introduced in Section 2.5. The system was decomposed into:

- Processes (CLS, SMGW, Backend)
- Data flows (CONNECT message, TLS handshake, application data)
- Trust boundaries (HAN, WAN, cryptographic boundary)

Each architecture was analysed separately.

- **Model 1 – Transparent TLS Proxy (CLI Mode)**

In the CLI architecture, the Controllable Local System (CLS) initiates a TCP connection toward the Smart Meter Gateway at the HAN interface. The SMGW acts as a mediation component and establishes a corresponding TCP connection toward the backend

system located in the WAN domain. Communication therefore follows the logical path CLS → SMGW → Backend.

However, the SMGW operates purely at the transport layer. The TLS session is negotiated directly between the CLS and the backend system after tunnel establishment. The SMGW forwards encrypted TLS records without terminating the session and does not possess cryptographic session keys. As a result, the cryptographic boundary remains strictly between CLS and backend, while the SMGW enforces mediation and authorization policies outside the TLS trust domain.

STRIDE Threat Table – Model 1 (CLI)

ID	Threat	STRIDE	Affected Part	Severity	Mitigation	Status
M1-1	Backend impersonation	Spoofing	TLS Handshake	Medium	Mutual TLS, CA validation	Mitigated
M1-2	CLS impersonation	Spoofing	Backend	Medium	Client certificate validation	Mitigated
M1-3	Unauthorized backend target	Elevation of Privilege	CONNECT Phase	Medium	Backend whitelist	Partially mitigated
M1-4	TLS record modification	Tampering	TLS Channel	Low	TLS integrity protection	Mitigated
M1-5	TLS handshake flooding	DoS	SMGW	Medium	No rate limiting	Open
M1-6	Private key compromise	Information Disclosure	CLS/Backend	Medium	Secure key storage	Device-dependent
M1-7	Insufficient logging	Repudiation	All	Medium	Logging limited	Open

M1-8	Payload inspection attempt	Information Disclosure	Application Data	Low	TLS not terminated	Mitigated

Security Interpretation – Model 1

In the CLI model, the TLS session is negotiated end-to-end between the CLS and the backend system. The Smart Meter Gateway operates purely as a transport-level mediation component, and forwards encrypted TLS records without accessing session keys. As a result, the cryptographic boundary remains strictly between CLS and backend.

Confidentiality of application data is therefore preserved even if the SMGW is compromised. Since TLS integrity protection applies to each record, any modification of encrypted payload during forwarding would be detected at the backend during decryption. The intermediary cannot manipulate application data without breaking the TLS integrity checks.

The identified residual risks are primarily related to availability and endpoint security rather than architectural weaknesses. For example, TLS handshake flooding may impact availability because rate limiting is not implemented. Additionally, compromise of endpoint private keys would undermine authentication guarantees independently of the proxy architecture.

Overall, the CLI model maintains a limited trusted computing base and preserves end-to-end confidentiality and integrity. From a security perspective, this architecture provides strong protection against information disclosure and tampering while satisfying the mediation requirement defined for the SMGW.

- **Model 2 – Transparent TLS Proxy (SRV Mode)**

In the SRV architecture, the direction of connection initiation is reversed compared to the CLI model. The Smart Meter Gateway actively establishes a TCP connection toward a service port exposed by the CLS within the Home Area Network. After the control phase and successful backend validation, the SMGW establishes a corresponding TCP connection toward the backend system in the WAN domain.

Although the initiation logic differs, the cryptographic properties remain unchanged. The TLS session is negotiated directly between the CLS and the backend system after tunnel establishment. The SMGW forwards encrypted TLS records without terminating the session and does not access session keys. Consequently, the cryptographic boundary remains strictly between CLS and backend.

The only structural difference compared to the CLI model is that the CLS exposes a listening service port, which slightly increases the externally visible attack surface. However, once TLS negotiation begins, mutual authentication ensures that only authorized entities can establish a valid secure channel.

STRIDE Threat Table – Model 2 (SRV)

ID	Threat	STRIDE	Affected Part	Severity	Mitigation	Status
M2-1	Unauthorized access to CLS port	Spoofing	TCP Service Port	Medium	TLS enforced after connection	Open
M2-2	Backend impersonation	Spoofing	TLS Handshake	Medium	Mutual TLS	Mitigated
M2-3	Fake CONNECT injection	Elevation of Privilege	CONNECT Phase	Medium	Whitelist validation	Partially mitigated
M2-4	TLS record modification	Tampering	TLS Channel	Low	TLS integrity	Mitigated
M2-5	Port flooding	DoS	CLS	Medium	No rate limiting	Open
M2-6	Private key compromise	Information Disclosure	CLS/Backend	Medium	Secure key storage	Device-dependent
M2-7	Insufficient logging	Repudiation	All	Medium	Logging limited	Open
M2-8	Payload inspection attempt	Information Disclosure	Application Data	Low	No TLS termination	Mitigated

Security Interpretation – Model 2

The SRV model preserves the same cryptographic properties as the CLI architecture. Although the direction of TCP connection initiation is reversed, the TLS handshake is still negotiated end-to-end between CLS and backend. The SMGW forwards encrypted TLS records without terminating the session and does not possess session keys.

As in the CLI model, confidentiality and integrity are preserved end-to-end. Any attempt to modify TLS records during transport would be detected by the TLS integrity mechanisms at the receiving endpoint. Plaintext application data is never exposed within the SMGW process.

However, the SRV model introduces a slightly larger attack surface because the CLS exposes a listening service port. Unauthorized connection attempts or port flooding may increase availability risks. Nevertheless, once TLS negotiation begins, mutual authentication ensures that unauthorized entities cannot establish a valid secure channel.

The trusted computing base remains limited to the actual TLS endpoints. Compared to CLI mode, the cryptographic guarantees are identical, but operational exposure is marginally increased due to the open service port. Despite this, the SRV model still maintains strong security properties and preserves end-to-end confidentiality and integrity.

- **Model 3 – TLS Termination Prototype**

In the TLS termination prototype, the Smart Meter Gateway operates as a cryptographic endpoint on both communication segments. The CLS establishes a TLS session directly with the SMGW at the HAN interface. After terminating this session, the SMGW establishes a second, independent TLS session toward the backend system in the WAN domain. The logical communication path can therefore be described as CLS → TLS → SMGW → TLS → Backend.

Unlike the transparent proxy models, the SMGW decrypts application data received from the CLS before re-encrypting it for transmission to the backend. As a result, the SMGW becomes part of the cryptographic boundary and has access to application payload in plaintext form.

This architectural decision significantly expands the trusted computing base, as the confidentiality and integrity of application data now depend on the correct and secure operation of the SMGW process itself. While TLS protection remains active on both segments, end-to-end cryptographic guarantees between CLS and backend are no longer preserved.

STRIDE Threat Table – Model 3 (Termination)

ID	Threat	STRIDE	Affected Part	Severity	Mitigation	Status
M3-1	Plaintext exposure at SMGW	Information Disclosure	SMGW	High	Architectural redesign required	Open
M3-2	Payload modification	Tampering	SMGW	High	Application-layer signing needed	Open
M3-3	Command injection via SMGW	Elevation of Privilege	SMGW	High	Not mitigated	Open
M3-4	Backend cannot verify origin	Repudiation	Backend	Medium	End-to-end signing required	Open
M3-5	Dual TLS handshake load	DoS	SMGW	Medium	No rate limiting	Open
M3-6	Private key compromise	Spoofing	All	Medium	Mutual TLS	Mitigated

Security Interpretation – Model 3

In the TLS termination model, the Smart Meter Gateway acts as a cryptographic endpoint on both communication segments. The TLS session from the CLS is terminated at the SMGW, and a second independent TLS session is established toward the backend. This architectural decision shifts the cryptographic boundary into the SMGW process. As a consequence, application data is decrypted within the SMGW and becomes available in plaintext. If the SMGW were compromised, confidentiality would be directly violated. Unlike the transparent proxy models, the intermediary now has full capability to inspect, modify, or inject payload data.

Integrity guarantees are also weakened. While TLS protects each individual segment, the backend cannot cryptographically verify that the received payload originated unchanged from the CLS. Trust must therefore be placed in the SMGW as an intermediary, which significantly expands the trusted computing base.

Furthermore, the presence of two independent TLS handshakes increases processing overhead and may introduce additional attack surfaces related to denial-of-service conditions.

Although TLS remains technically present on both segments, the overall security posture is significantly weaker compared to the transparent proxy architectures. The termination model introduces a single point of failure and increases both information disclosure and tampering risks.

6.3 Experimental Validation of Confidentiality

To verify that the SMGW does not access application payload in the transparent proxy models, both implementation-level logging and packet-level inspection were conducted.

- **Model 1 – CLI Proxy**

Application-Level Verification

The CLS transmits:

cls.py
(test) Meter value = 123.45 kWh

The SMGW forwarding logic:

Smgw.py
data = src.recv(4096) dst.sendall(data)

Observed output: Binary TLS records only and no readable plaintext, which confirms that SMGW does not possess TLS session keys.

Network-Level Verification (Wireshark Capture)

Traffic was captured on the loopback interface during execution of the CLI transparent proxy model. The packet trace illustrates both the transport-layer setup and the TLS handshake process. Initially, a standard TCP three-way handshake is observed (SYN, SYN-ACK, ACK) between the client-side ephemeral port and the SMGW listening port (8443). This establishes the transport-layer connection required for tunnel mediation.

Following TCP establishment, a TLSv1.3 ClientHello message is transmitted. This message contains the proposed TLS version, supported cipher suites, and key exchange parameters. The corresponding ServerHello message is returned by the backend system

(forwarded transparently by the SMGW), selecting cryptographic parameters and continuing the key agreement process.

After the Change Cipher Spec transition, subsequent handshake messages are encrypted in accordance with TLS 1.3. In the capture, these encrypted records appear as TLSv1.3 Application Data.

Once the handshake is completed, all transmitted payload frames are labelled as TLSv1.3 Application Data. No readable application strings such as:

“Meter value = 123.45 kWh”

appear anywhere in the packet trace. This confirms that application data is encrypted end-to-end between CLS and backend. The absence of plaintext payload at packet level demonstrates that the SMGW does not terminate TLS in CLI mode and does not access cryptographic session keys. The gateway forwards encrypted TLS records transparently while remaining outside the cryptographic boundary.

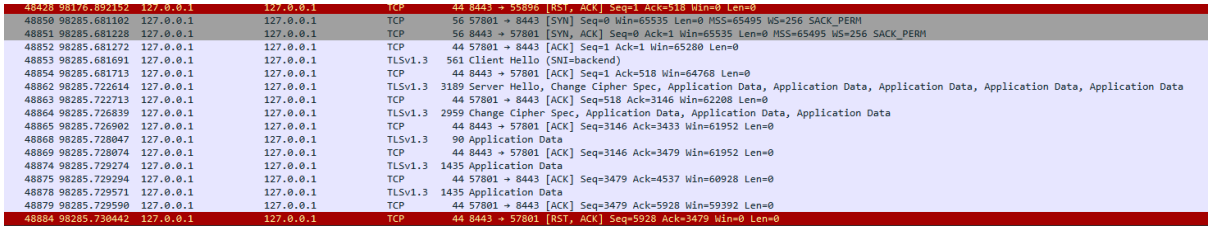


Figure 12: Wireshark capture of CLI proxy communication. After TLS handshake completion, only TLSv1.3 “Application Data” records are visible. No readable application payload appears in the packet trace, confirming that the SMGW does not access plaintext data.

- **Model 2 – SRV Proxy**

The verification results for the SRV model confirm that the Smart Meter Gateway does not create or manage any SSL context during communication. No wrap_socket() invocation is executed at the SMGW, which demonstrates that TLS termination does not occur at the intermediary.

Both packet-level inspection and console-level logging show that only encrypted TLS records are forwarded. The SMGW processes raw TCP buffers without interpreting or decrypting TLS payload. No readable application data appears in the console output during forwarding. These observations confirm that the SRV model preserves the same cryptographic boundary as the CLI architecture. The TLS session remains strictly end-to-end between CLS and backend, and the SMGW operates exclusively as a transport-layer mediation component.

- **Model 3 – TLS Termination Prototype**

The SMGW executes:

Smgw.py snippet code
data = cls_tls.recv(4096) print("SMGW DECRYPTED from CLS:", data.decode())

Observed console output:

SMGW DECRYPTED from CLS: PLAINTEXT FROM CLS

This confirms that SMGW terminates TLS and the payload visible in plaintext

Packet capture shows two independent TLS sessions:

1. CLS ↔ SMGW
2. SMGW ↔ Backend

No single end-to-end TLS session exists.

6.4 Comparative Architectural Evaluation

Cryptographic Boundary Comparison

Property	**CLI**	**SRV**	**Termination**
End-to-End TLS	Yes	Yes	No
SMGW Terminates TLS	No	No	Yes
Plaintext at SMGW	No	No	Yes
Trusted Computing Base	Limited	Limited	Expanded

Security Impact Comparison

Security Aspect	**CLI**	**SRV**	**Termination**
Tampering Risk	Low	Low	High
Disclosure Risk	Low	Low	High
Backend Verifies Original Integrity	Yes	Yes	No

Final Evaluation

The combined STRIDE assessment and experimental validation demonstrate:

1. End-to-end TLS is the decisive security property.
2. Transparent proxy models preserve confidentiality and integrity.
3. TLS termination expands the trusted computing base.
4. Compromise of SMGW in CLI or SRV does not expose application payload.
5. Compromise of SMGW in the termination model enables full data manipulation.

The results confirm that the transparent TLS proxy architecture aligned with the BSI Technical Guideline provides significantly stronger security guarantees than TLS termination at the intermediary.

7. Summary

This thesis investigated the implementation and evaluation of a secure communication channel for smart metering systems based on the TLS proxy concept. The objective was to answer the guiding question:

“How can a TLS channel setup with the Smart Meter Gateway acting as a proxy be implemented to guarantee end-to-end security and reliable performance between control box and grid operation backend?”

To address this question, a controlled reference implementation was developed in Python simulating three core components:

1. Controllable Local System (CLS)
2. Smart Meter Gateway (SMGW)
3. Backend server

Three architectural models were designed, implemented, and evaluated:

1. TLS Proxy – CLI Mode
2. TLS Proxy – SRV Mode
3. TLS Termination Prototype

The CLI and SRV models preserve end-to-end TLS between CLS and backend while the SMGW operates as a transparent mediation component. The termination model splits the TLS session and performs decryption at the SMGW.

A structured STRIDE-based threat analysis was conducted for each model. The analysis systematically evaluated risks related to spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege.

In contrast, the TLS termination model introduces:

- Plaintext exposure at the SMGW
- Expanded trusted computing base
- Increased tampering and privilege escalation risk
- Loss of verifiable end-to-end integrity

The comparative evaluation demonstrates that preserving end-to-end TLS significantly reduces high-impact threats in the categories of information disclosure, tampering, and elevation of privilege.

The findings confirm that a transparent TLS proxy architecture aligned with BSI provides stronger security guarantees than an intermediary termination approach while maintaining controlled mediation between HAN and WAN domains.

Integration Perspective: Application in GN3

Based on the initial exposé and project context, the presented architecture can also be transferred to large-scale grid environments such as GN3-based infrastructures.

Within a GN3 context, the SMGW would continue to act as the regulated security anchor at the boundary between local energy assets and higher-level grid control systems. The transparent TLS proxy approach could be implemented as follows:

- The SMGW enforces certificate-based mutual authentication for all connected control devices.
- End-to-end TLS between distributed control units and central GN3 control services is preserved.
- GN3 backend systems validate device identities using a regulated PKI model.
- Network segmentation and trust boundaries remain strictly enforced at gateway level.

A practical limitation of this work was related to the simulation environment. For a realistic emulation of the Smart Meter Gateway network topology, the use of GNS3 was considered. GNS3 is primarily designed to integrate virtual network devices and Linux-based virtual machines into complex network simulations [**Error! Reference source not found.**].

However, the available development environment was based on Microsoft Windows. While GNS3 is supported on Windows, advanced configurations such as lightweight Linux containers, native TAP interfaces, and certain routing features typically require a Linux host system or a fully configured Linux virtual machine.

Because no dedicated Linux machine was available, full-scale network emulation within GNS3 could not be implemented. As a result, the implementation was executed in a local Windows-based environment and all components (CLS, SMGW, Backend) were simulated using localhost communication.

This limitation does not affect the correctness of the TLS proxy implementation itself. However, it restricts the ability to evaluate network-layer aspects such as routing behaviour, firewall enforcement, and multi-interface separation in a fully virtualized infrastructure

8. References

1. Bundesamt für Sicherheit in der Informationstechnik, “Technische Richtlinie TR-03109-5: Detailspezifikationen,” BSI, 2023. [Online]. Available: https://www.bsi.bund.de/DE/Themen/Unternehmen-und-Organisationen/Standards-und-Zertifizierung/Smart-metering/Kommunikationsadapter/TechnRichtlinie/TR_03109-5_Detailspezifikationen.html. [Accessed: 10-Oct-2025].

2. Bundesamt für Sicherheit in der Informationstechnik, “Technische Richtlinie TR-03109-5: Implementierungshinweis,” BSI, 2023. [Online]. Available: https://www.bsi.bund.de/DE/Themen/Unternehmen-und-Organisationen/Standards-und-Zertifizierung/Smart-metering/Kommunikationsadapter/TechnRichtlinie/TR_03109-5_Implementierungshinweis.html. [Accessed: 10-Oct-2025].

3. Bundesamt für Sicherheit in der Informationstechnik, “Technische Richtlinie TR-03109-5: Kommunikationsadapter,” BSI, Nov. 24, 2023. [Online]. Available: https://www.bsi.bund.de/DE/Themen/Unternehmen-und-Organisationen/Standards-und-Zertifizierung/Smart-metering/Kommunikationsadapter/TechnRichtlinie/TR_03109-5_node.html. [Accessed: 10-Oct-2025].

4. GeeksforGeeks, “Transport Layer Security (TLS),” Nov. 21, 2025. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/transport-layer-security-tls/. [Accessed: 03-Dec-2025].

5. GeeksforGeeks, “What is Proxy Server?”, Nov. 21, 2025. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/what-is-proxy-server/. [Accessed: Dec. 03, 2025].

6. GNS3 Technologies Inc., “GNS3 – Network Emulator Software,” GNS3, 2026. [Online]. Available: https://gns3.com/software. [Accessed: Feb. 25, 2026].

7. F. Swiderski and W. Snyder, *Threat Modeling*, Redmond, WA, USA: Microsoft Press, 2004. [Online]. Available: https://www.microsoftpressstore.com/store/threat-modeling-9780735619913. [Accessed: 23-Feb-2026].

Appendix

Model 1: CLI implementation

Backend.py

```
"""
Backend Server
--------------
This component represents the WAN communication partner
the backend is the real TLS endpoint.
TLS handshake terminates here.
The SMGW only forwards encrypted TLS records and does NOT
possess the session keys.

"""

import socket
import ssl

HOST = "127.0.0.1"
PORT = 9443  # Simulated WAN TLS service port

# Create TLS context expecting client authentication
# (Mutual TLS as required by TR secure communication model)
ctx = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)

# Load CA to verify CLS certificate
ctx.load_verify_locations("certs/ca.pem")
ctx.verify_mode = ssl.CERT_REQUIRED

# Load backend server certificate and private key
ctx.load_cert_chain(
   certfile="certs/backend.pem",
   keyfile="certs/backend.key"
)

# Create TCP socket
sock = socket.socket()
sock.bind((HOST, PORT))
sock.listen(5)

print("Backend: waiting for CLS via SMGW...")

# Accept incoming connection from SMGW tunnel
conn, _ = sock.accept()
```

```
# Perform TLS handshake
# IMPORTANT: This handshake is end-to-end between CLS and Backend.
tls = ctx.wrap_socket(conn, server_side=True)

print("Backend: End-to-end TLS handshake successful")

# Receive application data (decrypted here)
data = tls.recv(4096)
print("Backend received:", data.decode())

tls.close()
sock.close()
```

CLS.py

```
"""
CLS Client (HKS.TLSPROXY.CLI)
-----------------------------

This component represents the Control Box (CLS).

Sequence:
1. Connect to SMGW (HAN interface)
2. Send CONNECT request (control phase)
3. Establish end-to-end TLS through the tunnel
4. Send application data securely

TLS handshake is performed directly with the Backend.
SMGW only forwards encrypted TLS records.
"""

import socket
import ssl
import time

SMGW_HOST = "127.0.0.1"
SMGW_PORT = 8443

BACKEND_HOST = "127.0.0.1"
BACKEND_PORT = 9443

# ========================
# STEP 1: CONNECT TO SMGW
# ========================
sock = socket.create_connection((SMGW_HOST, SMGW_PORT))
```

```
# ========================
# STEP 2: SEND CONNECT REQUEST
# ========================
# This simulates the CLI proxy channel establishment
# as described in TR-03109-5.

connect_msg = f"CONNECT {BACKEND_HOST} {BACKEND_PORT}"
sock.sendall(connect_msg.encode())

print("CLS: Proxy CONNECT request sent")

# ========================
# STEP 3: START END-TO-END TLS
# ========================
# TLS handshake will pass through SMGW unchanged.

ctx = ssl.create_default_context(ssl.Purpose.SERVER_AUTH)
ctx.load_verify_locations("certs/ca.pem")
ctx.verify_mode = ssl.CERT_REQUIRED
ctx.check_hostname = False

# Load CLS certificate for mutual authentication
ctx.load_cert_chain(
   certfile="certs/cls.pem",
   keyfile="certs/cls.key"
)

# Wrap the socket to start TLS handshake
tls = ctx.wrap_socket(sock, server_hostname="backend")

print("CLS: End-to-end TLS handshake successful")

# ========================
# STEP 4: SEND APPLICATION DATA
# ========================
# This is encrypted end-to-end.
# SMGW cannot read this payload.

tls.sendall(b"(test) Meter value = 123.45 kWh")

time.sleep(1)
tls.close()
```

SMGW_proxy.py

```
"""
SMGW TLS-Proxy (CLI Scenario)
-----------------------------

This component models the TLS-Proxy function of the SMGW
as defined in TR-03109-5 (HKS.TLSPROXY.CLI).

IMPORTANT:
- SMGW does NOT terminate the application TLS session.
- SMGW performs mediation and tunnel establishment.
- TLS session keys are NOT known to SMGW.

Structure:
1. Control phase (CONNECT request)
2. Authorization
3. Tunnel establishment
4. Forwarding encrypted TLS records
"""

import socket
import threading

LISTEN_HOST = "127.0.0.1"
LISTEN_PORT = 8443  # Simulated IF_GW_CLS interface port

def forward(src, dst):
  """
  Forwards raw TCP data between two sockets.

  This function forwards encrypted TLS records
  without inspecting or modifying them.

  This preserves end-to-end confidentiality.
  """
  try:
    while True:
      data = src.recv(4096)
      if not data:
        break
      print(data)
      dst.sendall(data)
  except:
    pass
  finally:
    src.close()
    dst.close()
```

```
# Create listening socket (HAN interface simulation)
listen_sock = socket.socket()
listen_sock.bind((LISTEN_HOST, LISTEN_PORT))
listen_sock.listen(5)

print("SMGW: Listening for CLI proxy requests...")

# Accept connection from CLS
cls_conn, addr = listen_sock.accept()
print("SMGW: CLS connected from", addr)

# =========================
# CONTROL PHASE
# =========================
# CLS sends a CONNECT request specifying target backend.
# This simulates HKS.TLSPROXY.CLI channel establishment.

request = cls_conn.recv(1024).decode()

if not request.startswith("CONNECT"):
   print("SMGW: Invalid proxy request")
   cls_conn.close()
   exit()

_, backend_host, backend_port = request.split()
backend_port = int(backend_port)

# Basic access control (demonstrates mediation role)
ALLOWED_BACKEND = "127.0.0.1"

if backend_host != ALLOWED_BACKEND:
   print("SMGW: Unauthorized backend target")
   cls_conn.close()
   exit()

print(f"SMGW: Authorizing tunnel to {backend_host}:{backend_port}")

# =========================
# TUNNEL ESTABLISHMENT
# =========================
# SMGW opens TCP connection to WAN partner.

backend_conn = socket.create_connection((backend_host, backend_port))

print("SMGW: Tunnel established, switching to forwarding mode")
```

```
# ========================
# DATA PHASE
# ========================
# From here onward, only encrypted TLS records are forwarded.
# SMGW does not decrypt and does not possess TLS keys.

threading.Thread(target=forward, args=(cls_conn, backend_conn)).start()
threading.Thread(target=forward, args=(backend_conn, cls_conn)).start()
```

Model 2: SRV implementation

Backend.py

```
import socket
import ssl

HOST = "127.0.0.1"
PORT = 9443

ctx = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
ctx.load_verify_locations("certs/ca.pem")
ctx.verify_mode = ssl.CERT_REQUIRED

ctx.load_cert_chain(
   certfile="certs/backend.pem",
   keyfile="certs/backend.key"
)
sock = socket.socket()
sock.bind((HOST, PORT))
sock.listen(5)

print("Backend: waiting for connection...")

conn, _ = sock.accept()
print("Backend: TCP connection accepted")

tls = ctx.wrap_socket(conn, server_side=True)

print("Backend: End-to-end TLS handshake successful")

data = tls.recv(4096)
print("Backend received:", data.decode())

tls.close()
sock.close()
```

cls.py

```
import socket
import ssl
import time

HOST = "127.0.0.1"
PORT = 8555  # CLS service port

sock = socket.socket()
sock.bind((HOST, PORT))
sock.listen(1)

print("CLS (SRV): Waiting for SMGW connection...")

conn, _ = sock.accept()
print("CLS: SMGW connected")

# =========================
# Send CONNECT request
# =========================
connect_msg = "CONNECT 127.0.0.1 9443"
conn.sendall(connect_msg.encode())

# =========================
# Now start end-to-end TLS
# =========================
ctx = ssl.create_default_context(ssl.Purpose.SERVER_AUTH)
ctx.load_verify_locations("certs/ca.pem")
ctx.verify_mode = ssl.CERT_REQUIRED
ctx.check_hostname = False

ctx.load_cert_chain(
   certfile="certs/cls.pem",
   keyfile="certs/cls.key"
)

tls = ctx.wrap_socket(conn, server_hostname="backend")

print("CLS: End-to-end TLS handshake successful")

tls.sendall(b"(SRV test) Meter value = 456.78 kWh")

time.sleep(1)
tls.close()
sock.close()
```

SMGW_proxy.py

```
import socket
import threading

CLS_HOST = "127.0.0.1"
CLS_PORT = 8555

def forward(src, dst):
  try:
    while True:
      data = src.recv(4096)
      if not data:
        break
      dst.sendall(data)
  except:
    pass
  finally:
    src.close()
    dst.close()

print("SMGW (SRV): Connecting to CLS...")

# =========================
# STEP 1: Connect to CLS
# =========================
cls_conn = socket.create_connection((CLS_HOST, CLS_PORT))
print("SMGW: Connected to CLS")

# =========================
# STEP 2: Receive CONNECT request
# =========================
request = cls_conn.recv(1024).decode()

if not request.startswith("CONNECT"):
  print("SMGW: Invalid request")
  cls_conn.close()
  exit()

_, backend_host, backend_port = request.split()
backend_port = int(backend_port)

print(f"SMGW: Connecting to backend {backend_host}:{backend_port}")

backend_conn = socket.create_connection((backend_host, backend_port))
print("SMGW: Tunnel established")
```

```
# ========================
# STEP 3: Forward encrypted TLS
# ========================
threading.Thread(target=forward, args=(cls_conn, backend_conn)).start()
threading.Thread(target=forward, args=(backend_conn, cls_conn)).start()
```

Output console

Backend output

```
PS C:\smart-meter-thesis\oldsrv> py .\backend_server.py
Backend: waiting for connection...
Backend: TCP connection accepted
Backend: End-to-end TLS handshake successful
Backend received: (SRV test) Meter value = 456.78 kWh
PS C:\smart-meter-thesis\oldsrv>
```

CLS output

```
PS C:\smart-meter-thesis\oldsrv> py .\cls_server.py
CLS (SRV): Waiting for SMGW connection...
CLS: SMGW connected
CLS: End-to-end TLS handshake successful
PS C:\smart-meter-thesis\oldsrv>
```

SMGW output:

```
PS C:\smart-meter-thesis\oldsrv> py .\smgw_proxy.py
SMGW (SRV): Connecting to CLS...
SMGW: Connected to CLS
SMGW: Connecting to backend 127.0.0.1:9443
SMGW: Tunnel established
PS C:\smart-meter-thesis\oldsrv>
```

Model 3: first try, old implementation

Backend.py

```
import socket, ssl

HOST = "127.0.0.1"
PORT = 9443

ctx = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
ctx.load_cert_chain("certs/backend.pem", "certs/backend.key")
ctx.load_verify_locations("certs/ca.pem")
ctx.verify_mode = ssl.CERT_REQUIRED

sock = socket.socket()
sock.bind((HOST, PORT))
sock.listen(1)

print("Backend: waiting for SMGW...")
```

```
conn, _ = sock.accept()
tls = ctx.wrap_socket(conn, server_side=True)

data = tls.recv(4096)
print("Backend received PLAINTEXT:", data.decode())

tls.sendall(b"ACK from backend")
tls.close()
```

cls.py

```
import socket, ssl

SMGW_ADDR = ("127.0.0.1", 8443)

ctx = ssl.create_default_context(ssl.Purpose.SERVER_AUTH)
ctx.load_cert_chain("certs/cls.pem", "certs/cls.key")
ctx.load_verify_locations("certs/ca.pem")

sock = socket.create_connection(SMGW_ADDR)
tls = ctx.wrap_socket(sock, server_hostname="smgw")

tls.sendall(b"PLAINTEXT FROM CLS")
print("CLS got:", tls.recv(4096).decode())

tls.close()
```

SMGW_proxy.py

```
import socket, ssl

CLS_PORT = 8443
BACKEND_ADDR = ("127.0.0.1", 9443)

# --- TLS server context (for CLS) ---
server_ctx = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
server_ctx.load_cert_chain("certs/smgw.pem", "certs/smgw.key")
server_ctx.load_verify_locations("certs/ca.pem")
server_ctx.verify_mode = ssl.CERT_REQUIRED

listen_sock = socket.socket()
listen_sock.bind(("127.0.0.1", CLS_PORT))
listen_sock.listen(1)

print("SMGW: waiting for CLS...")
```

```
cls_conn, _ = listen_sock.accept()
cls_tls = server_ctx.wrap_socket(cls_conn, server_side=True)

data = cls_tls.recv(4096)
print("SMGW DECRYPTED from CLS:", data.decode())

# --- TLS client context (to Backend) ---
client_ctx = ssl.create_default_context(ssl.Purpose.SERVER_AUTH)
client_ctx.load_cert_chain("certs/smgw.pem", "certs/smgw.key")
client_ctx.load_verify_locations("certs/ca.pem")

backend_sock = socket.create_connection(BACKEND_ADDR)
backend_tls = client_ctx.wrap_socket(backend_sock, server_hostname="backend")

backend_tls.sendall(data)
reply = backend_tls.recv(4096)

cls_tls.sendall(reply)

cls_tls.close()
backend_tls.close()
```

Output console

Backend output

```
root@Marshmallow:~/smart-meter-thesis/SRVfinal# python3 backend_server.py
Backend: waiting for SMGW...
Backend received PLAINTEXT: PLAINTEXT FROM CLS
root@Marshmallow:~/smart-meter-thesis/SRVfinal#
```

SMGW output

```
root@Marshmallow:~/smart-meter-thesis/SRVfinal# python3 smgw_proxy.py
SMGW: waiting for CLS...
SMGW DECRYPTED from CLS: PLAINTEXT FROM CLS
root@Marshmallow:~/smart-meter-thesis/SRVfinal#
```

CLS output

```
root@Marshmallow:~/smart-meter-thesis/SRVfinal# python3 cls_server.py
CLS got: ACK from backend
root@Marshmallow:~/smart-meter-thesis/SRVfinal#
```